

Технологии программирования

- Spring MVC-

Spring MVC

Spring MVC – проект (модуль), который обеспечивает архитектуру паттерна Model — View — Controller (Модель — Отображение (далее — Вид) — Контроллер) при помощи слабо связанных готовых компонентов. Паттерн MVC разделяет аспекты приложения (логику ввода, бизнес-логику и логику UI), обеспечивая при этом свободную связь между ними.

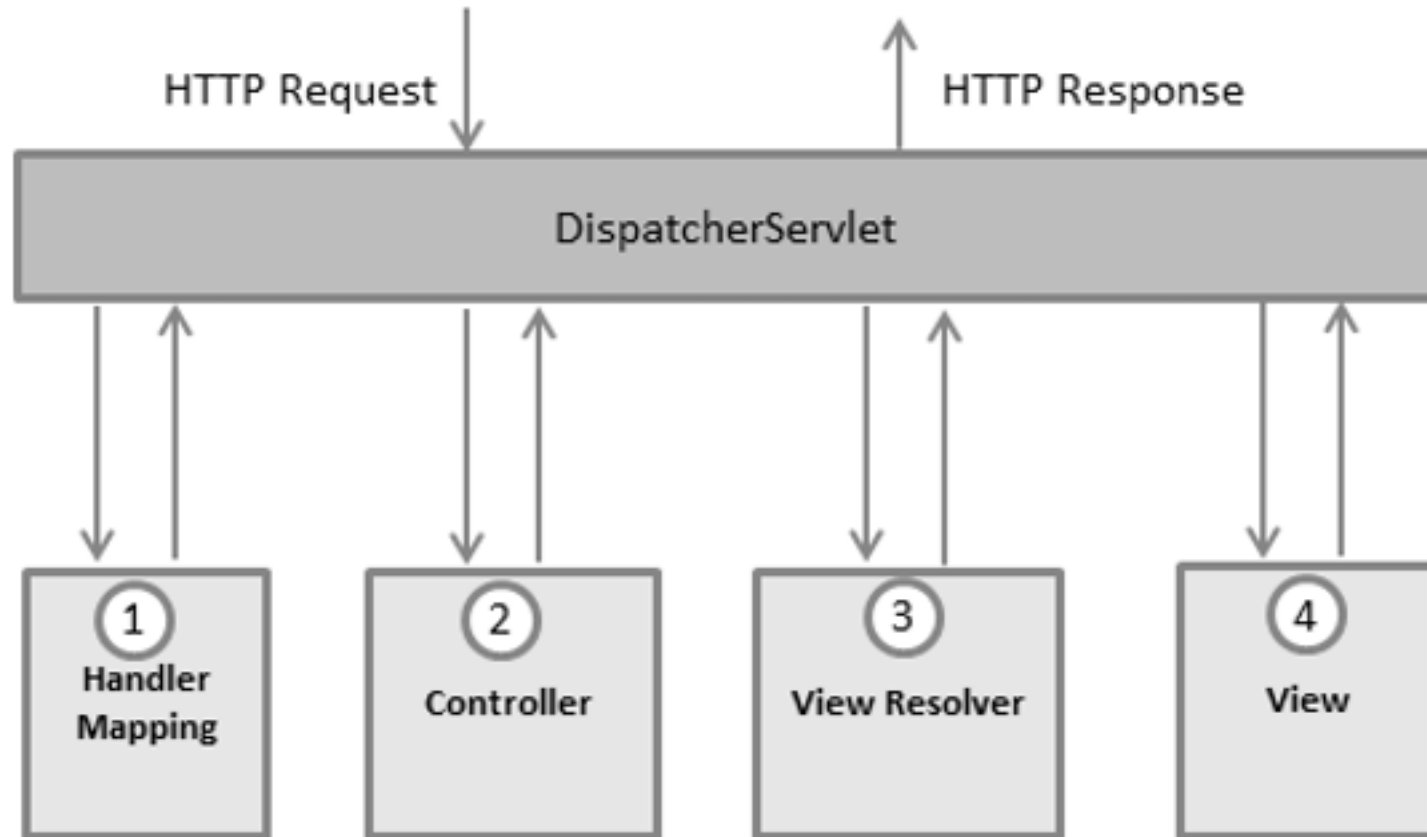
Spring MVC

- **Model** (Модель) инкапсулирует (объединяет) данные приложения, в целом они будут состоять из POJO («Старых добрых Java-объектов», или бинов).
- **View** (Отображение, Вид) отвечает за отображение данных Модели, — как правило, генерируя HTML, которые мы видим в своём браузере.
- **Controller** (Контроллер) обрабатывает запрос пользователя, создаёт соответствующую Модель и передаёт её для отображения в Вид.

DispatcherServlet

Spring MVC построен вокруг центрального сервлета, который распределяет запросы по контроллерам, а также предоставляет другие широкие возможности при разработке веб приложений. На самом деле DispatcherServlet — полностью интегрированный сервлет в Spring IoC контейнер и таким образом получает доступ ко всем возможностям Spring.

DispatcherServlet



DispatcherServlet

- После получения HTTP-запроса DispatcherServlet обращается к интерфейсу HandlerMapping, который определяет, какой Контроллер должен быть вызван, после чего, отправляет запрос в нужный Контроллер.
- Контроллер принимает запрос и вызывает соответствующий служебный метод, основанный на GET или POST. Вызванный метод определяет данные Модели, основанные на определённой бизнес-логике и возвращает в DispatcherServlet имя Вида (View).
- При помощи интерфейса ViewResolver DispatcherServlet определяет, какой Вид нужно использовать на основании полученного имени.
- После того, как Вид (View) создан, DispatcherServlet отправляет данные Модели в виде атрибутов в Вид, который в конечном итоге отображается в браузере.

@Controller

DispatcherServlet отправляет запрос контроллерам для выполнения определённых функций. Аннотация @Controller указывает, что конкретный класс является контроллером. Аннотация @RequestMapping используется для мапинга (связывания) с URL для всего класса или для конкретного метода обработчика.

@Controller

```
@Controller
@RequestMapping("/hello")
public class HelloController {
    @RequestMapping(method = RequestMethod.GET)
    public String printHello(ModelMap model) {
        model.addAttribute("message", "Hello Spring MVC Framework!");
        return "hello";
    }
}
```


Специальные bean Spring MVC

DispatcherServlet делегирует специальные компоненты для обработки запросов и вывода соответствующих ответов. Под «специальными bean-компонентами» мы подразумеваем управляемые Spring экземпляры Object, которые реализуют фреймворк-контракты. Обычно они поставляются со встроенными контрактами, но вы можете настроить их свойства и расширить или заменить их. В следующей таблице перечислены специальные bean-компоненты, обнаруженные DispatcherServlet:

Специальные bean Spring MVC

DispatcherServlet делегирует специальные компоненты для обработки запросов и вывода соответствующих ответов. Под «специальными bean-компонентами» мы подразумеваем управляемые Spring экземпляры Object, которые реализуют фреймворк-контракты. Обычно они поставляются со встроенными контрактами, но вы можете настроить их свойства и расширить или заменить их.

Spring MVC -HandlerMapping

Сопоставляет запрос с обработчиком вместе со списком перехватчиков для предварительной и последующей обработки. Сопоставление основано на некоторых критериях, детали которых зависят от реализации HandlerMapping. Двумя основными реализациями HandlerMapping являются RequestMappingHandlerMapping (который поддерживает аннотированные методы @RequestMapping) и SimpleUrlHandlerMapping (который поддерживает явную регистрацию шаблонов пути URI к обработчикам).

Spring MVC -HandlerAdapter

Помогает DispatcherServlet вызвать обработчик, сопоставленный с запросом, независимо от того, как фактически вызывается обработчик. Например, для вызова аннотированного контроллера требуется разрешающие аннотации. Основная цель HandlerAdapter — оградить DispatcherServlet от таких подробностей.

Spring MVC – другие бины

- `HandlerExceptionResolver` - стратегия разрешения исключений, возможное сопоставление их с обработчиками, представлениями ошибок HTML или другими целями.
- `ViewResolver` - преобразует логические имена представлений на основе строк, возвращенные обработчиком, в фактическое представление, с помощью которого выполняется рендеринг в ответ.
- `LocaleResolver`, `LocaleContextResolver` - определите «локаль», которую использует клиент, и, возможно, его часовой пояс, чтобы иметь возможность предлагать интернационализированные представления.

Spring MVC – другие бины

- ThemeResolver - определяет темы, которые может использовать ваше веб-приложение — например, чтобы предлагать персонализированные макеты.
- MultipartResolver - абстракция для разбора запроса, состоящего из нескольких частей (например, загрузки файла формы браузера) с помощью некоторой библиотеки разбора составных частей.
- FlashMapManager - хранит и извлекает «входную» и «выходную» FlashMap, которые можно использовать для передачи атрибутов из одного запроса в другой, обычно через перенаправление.

Spring MVC – как приходит запрос

1. После получения HTTP-запроса `DispatcherServlet` перебирает доступные ему (предварительно найденные в контексте) экземпляры `HandlerMapping`, один из которых определит, метод какого `Controller` должен быть вызван. Реализации `HandlerMapping`, использующиеся по умолчанию: `BeanNameUrlHandlerMapping` и `RequestMappingHandlerMapping` (создаёт экземпляры `RequestMappingInfo` по методам аннотированным `@RequestMapping` в классах с аннотацией `@Controller`). `HandlerMapping` по `HttpServletRequest` находит соответствующий обработчик — handler-объект (например, `HandlerMethod`). Каждый `HandlerMapping` может иметь несколько реализаций `HandlerInterceptor` — интерфейса для кастомизации пред- и постобработки запроса. Список из `HandlerInterceptor`'ов и handler-объекта образуют экземпляр класса `HandlerExecutionChain`, который возвращается в `DispatcherServlet`.

Spring MVC – как приходит запрос

2. Для выбранного обработчика определяется соответствующий HandlerAdapter из предварительно найденных в контексте. По умолчанию используются `HttpRequestHandlerAdapter` (поддерживает классы, реализующие интерфейс `HttpRequestHandler`), `SimpleControllerHandlerAdapter` (поддерживает классы, реализующие интерфейс `Controller`) или `RequestMappingHandlerAdapter` (поддерживает контроллеры с аннотацией `@RequestMapping`).

Spring MVC – как приходит запрос

3. Происходит вызов метода `applyPreHandle` объекта `HandlerExecutionChain`. Если он вернёт `true`, то значит все `HandlerInterceptor` выполнили свою предобработку и можно перейти к вызову основного обработчика. `false` будет означать, что один из `HandlerInterceptor` взял обработку ответа на себя в обход основного обработчика.

Spring MVC – как приходит запрос

4. Выбранный `HandlerAdapter` извлекается из `HandlerExecutionChain` и с помощью метода `handle` принимает объекты запроса и ответа, а также найденный метод-обработчик запроса.

Spring MVC – как приходит запрос

5. Метод-обработчик запроса из Controller (вызванный через handle) выполняется и возвращает в DispatcherServlet ModelAndView. При помощи интерфейса ViewResolver DispatcherServlet определяет, какой View нужно использовать на основании полученного имени. Если мы имеем дело с REST-Controller или RESTful-методом контроллера, то вместо ModelAndView в DispatcherServlet из Controller вернётся null и, соответственно, никакой ViewResolver задействован не будет — ответ сразу будет полностью содержаться в теле HttpServletResponse после выполнения handle. Чтобы определить RESTful-методы, достаточно аннотировать их @ResponseBody либо вместо @Controller у класса поставить @RestController, если все методы контроллера будут RESTful.



Spring MVC – как приходит запрос

6. Перед завершением обработки запроса у объекта `HandlerExecutionChain` вызывается метод `applyPostHandle` для постобработки с помощью `HandlerInterceptor`ов.

Spring MVC – как приходит запрос

7. Если в процессе обработки запроса выбрасывается исключение, то оно обрабатывается с помощью одной из реализаций интерфейса `HandlerExceptionHandlerResolver`. По умолчанию используются `ExceptionHandlerExceptionHandlerResolver` (обрабатывает исключения из методов, аннотированных `@ExceptionHandler`), `ResponseStatusExceptionHandlerResolver` (используется для отображения исключений аннотированных `@ResponseStatus` в коды HTTP-статусов) и `DefaultHandlerExceptionHandlerResolver` (отображает стандартные исключения Spring MVC в коды HTTP-статусов).

Spring MVC – как приходит запрос

8. В случае с классическим Controller после того, как View создан, DispatcherServlet отправляет данные в виде атрибутов в View, который в конечном итоге записывается в HttpServletResponse. Для REST-Controller ответ данная логика не вызывается, ведь ответ уже в HttpServletResponse.

Spring MVC – как приходит запрос (еще чуть-чуть.....)

Когда HTTP запрос приходит с указанным заголовком Accept, Spring MVC перебирает доступные `HttpMessageConverter` до тех пор, пока не найдет того, кто сможет конвертировать из типов POJO доменной модели в указанный тип заголовка Accept. `HttpMessageConverter` работает в обоих направлениях: тела входящих запросов конвертируются в Java объекты, а Java объекты конвертируются в тела HTTP ответов.

По умолчанию, Spring Boot определяет довольно обширный набор реализаций `HttpMessageConverter`, подходящие для использования широкого круга задач, но также можно добавить поддержку и для других форматов в виде собственной или сторонней реализации `HttpMessageConverter` или переопределить существующие.

